Performance Analysis Report: Smoke vs. Load Testing

1. Overview

The purpose of this performance analysis is to convey the stability, reliability, and responsiveness of the DummyJSON Api, targeting exactly three endpoints.

The evaluation was conducted over two stages: a Smoke Test to quickly verify the endpoints are correctly functioning under minimal load, and a Load Test to observe the behavioral scenario under multiple concurrent users. All in order to identify response time, thresholds, error rates and bottlenecks if any.

2. Targeted Api Endpoints

- Get/ cart (list) -> retrieves a list of all cart records.
- Get/ products/ {validID} (single product) -> fetches detailed information about a certain product by its id.
- Get/ users/ search -> excutes a filtered search for user's profiles based on query parameter (e.g., searchName = "Emilys")

3. Detailed Technical Analysis

➢ Smoke Test

The system shows great stability and responsiveness under minimal load along with the consistent low response time. The insignificant different between average time and **P(95)** reflects a reliable performance with no bottlenecks.

- **Observations:**
  1. Conducted with **3 Virtual Users** over a **30-second** duration.
  2. Achieved a **100% success rate**.
  3. A total of **489 HTTP requests** were processed.
  4. The performance was highly efficient, with an average of **174.2 ms**.
  5. **90%** of requests completed within **178.5 ms (P90)** and **95%** within **205.6 ms (P95)**, indicating stable and predictable performance.

- **Load Test**

The System is functionally stable, yet the **Maximum Response Time** has spiked compared to the smoke test, which lead to higher concurrency. However the **Average Response Time** and **P(90**) remained acceptable, the **P(95)** and **Maximum Response Time** sharp increase indicates connection overhead as a major factor in performance degradation.

- **Observations:**
  1. Conducted with **20 Virtual Users** over a **30-second** duration.
  2. Achieved a **100% success rate**.
  3. A total of **501 HTTP requests** were processed.
  4. The load was handled efficiently, with an average response time of **220.3 ms**.
  5. **90%** of requests completed within **202.3 ms (P90)** and **95%** rose to **419.4 ms (P95)**, indicating increased response time variability under load.

4. Performance Comparison Table

Metric	**Smoke Test**	**Load Test**	**Comparison / Change**
Users	3 VUs	20 VUs	+566.7%
Total Requests	489	501	+2.45%
Throughput	16.13 req/s	14.92 req/s	-7.5%, efficiency dropped slightly as users increased.
Minimum Response Time	149.45 ms	150.84 ms	+0.9%, fastest responses remained stable.

Metric	Smoke Test	Load Test	Comparison / Change
Avg. Response Time	174.26 ms	220.32 ms	+26.4%, it rose up under load
Max Response Time	592.96 ms	4324.98 ms	+629.4% Spike: outliers appeared under load.
P(90) Response Time	178.64 ms	202.33 ms	+13.3%
P(95) Response Time	205.65 ms	419.44 ms	+103.9%, it doubled for the slowest 5% of users
Checks (Pass/Fail)	2608 / 0	2672 / 0	100% Pass in both
Success Rate	100%	100%	No change
Failure Rate	0%	0%	No change

5. Bottleneck Identification

- The **Maximum Response Time** climb by over 600% shows that the system is hitting a limit affecting a small percentage of requests severely.
- Under the load of 20 Vus, the system struggled to create a secure connection which took around 1.6 seconds.
- There was an increase in http_req_blocked (avg 37.15 ms) suggests thar requests are being queued waiting for an available slot.

6. Likely Causes

- The **limited number** of worker threads the system may have, that when **20 requests** hit simultaneously some of them are placed in a **wait** state.
- Since testing a free REST Api e.g**. DummyJSON**, the spikes could be caused by external rate-limiting considering the load test as a potential **DOS** attacks and deliberately deny responses.

7. Practical Improvement Recommendation

- **Circuit Breaker Pattern:** If a specific service (like the "Users" search) starts taking 4 seconds to respond, a "Circuit Breaker" will automatically stop trying to hit that service for a short period and return a fallback message. This prevents one slow endpoint from dragging down the entire system.
- **Caching Strategy:** Implementing a cache layer for frequent requests to produce processing time for requests, allowing it to clear request queue faster.
- **Auto-Scaling:** Configure your cloud provider (AWS/Azure) to automatically add more CPU and Memory the moment it detects a spike in CPU utilization.